

COMSATS UNIVERSITY ISLAMABAD



MACHINE LEARNING PROJECT

FINAL REPORT

NYC Taxi Trip Duration Classification

Submitted by:

Daniyal Nadeem FA24-BBD-120

1. PROJECT OVERVIEW

This project focuses on classifying NYC taxi trip durations using real-world taxi data sourced from the NYC Taxi and Limousine Commission. In the transportation industry, accurately predicting trip duration is valuable for dispatching, pricing, and fleet management. If a system can classify a trip as short, medium, or long at the time of booking, it enables smarter routing decisions and more transparent fare estimates for passengers.

The project applies a machine learning approach, specifically Logistic Regression combined with Principal Component Analysis (PCA), to classify trips into three duration categories. The complete workflow covers data loading and sampling, null value handling, outlier detection and removal, exploratory data analysis, feature engineering, categorical encoding, dimensionality reduction, model training, evaluation, and an interactive Gradio frontend.

Project Objectives

- Load and sample a 100,000+ row NYC taxi dataset for computational efficiency.
- Perform null value checks and apply appropriate imputation strategies.
- Detect and remove outliers and invalid entries using multiple statistical methods.
- Engineer temporal features from pickup datetime to enrich the feature space.
- Perform exploratory data analysis through six targeted visualizations.
- Apply Label Encoding to the target variable and One-Hot Encoding to categorical features.
- Reduce dimensionality with PCA retaining 95% of explained variance.
- Train a Logistic Regression classifier on PCA-reduced features.
- Evaluate model performance using accuracy, precision, recall, F1-score, and confusion matrix.
- Benchmark Logistic Regression against Random Forest on the same train-test split.
- Deploy an interactive prediction UI using Gradio.

2. DATASET DESCRIPTION

The dataset used in this project is the NYC Taxi Trip Duration dataset, loaded as NYC.csv. It contains anonymized records of individual taxi trips across New York City, including pickup and dropoff coordinates, timestamps, passenger counts, and trip durations. The full dataset exceeds 100,000 rows and 11 columns. A reproducible sample of 100,000 rows is drawn using a fixed random seed to ensure computational efficiency without sacrificing statistical representativeness.

Key Columns in the Dataset

Column Name	Description
trip_duration	Target variable: duration of the taxi trip in seconds.
pickup_datetime	Timestamp of when the passenger was picked up.
passenger_count	Number of passengers in the vehicle.
vendor_id	Identifier for the taxi technology provider (1 or 2).
store_and_fwd_flag	Whether the trip record was stored before sending (Y or N).
pickup_longitude / latitude	GPS coordinates of the trip origin.
dropoff_longitude / latitude	GPS coordinates of the trip destination.
hour	Engineered: hour of pickup extracted from pickup_datetime.
day_of_week	Engineered: day of the week (0=Monday, 6=Sunday).
month	Engineered: month of the year extracted from pickup_datetime.

The dataset contains over 100,000 rows, providing sufficient volume to produce statistically reliable machine learning results. After outlier removal and invalid entry filtering, the usable dataset remains large enough for robust model training and evaluation.

3. LIBRARY IMPORTS

The first step in any Python data science project is importing the required libraries. These are pre-built tools that handle data manipulation, visualization, and machine learning without writing everything from scratch.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

```
# ML Libraries
from sklearn.preprocessing import OneHotEncoder, LabelEncoder, StandardScaler
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
```

```
classification_report, confusion_matrix, ConfusionMatrixDisplay
)
```

```
import joblib
import gradio as gr
```

```
sns.set_theme(style='whitegrid', palette='muted')
```

Explanation of Each Library

Library	Purpose
pandas	Provides the DataFrame structure for loading, cleaning, and manipulating tabular data.
numpy	Handles mathematical operations on arrays and matrices efficiently.
matplotlib.pyplot	Base library for creating all kinds of plots and charts.
seaborn	Built on matplotlib; provides cleaner, more attractive statistical visualizations.
scipy.stats	Used for Z-score computation during outlier detection.
LabelEncoder	Converts ordered text categories (short, medium, long) into numeric labels for the target column.
OneHotEncoder	Creates binary columns for each category level in nominal feature columns.
StandardScaler	Scales numeric features to zero mean and unit variance before PCA.
PCA	Reduces the number of input features by projecting data onto principal components.
train_test_split	Splits the dataset into training and testing portions using stratified sampling.
LogisticRegression	Primary classification algorithm used to predict trip duration category.
RandomForestClassifier	Benchmark model for comparing against Logistic Regression.

Library	Purpose
confusion_matrix / ConfusionMatrixDisplay	Produces and visualizes the matrix of correct and incorrect predictions per class.
joblib	Saves trained model objects to disk for use in the Gradio frontend.
gradio	Builds the interactive web-based prediction UI for end users.

4. DATA LOADING AND EXPLORATION

After importing libraries, the full dataset is loaded from a CSV file into a pandas DataFrame. A reproducible sample of 100,000 rows is then drawn to balance computational efficiency with statistical richness. Several exploration commands are applied to understand the data before any cleaning begins.

```
# Load full dataset
df = pd.read_csv("NYC.csv")
print(f'Full dataset rows: {len(df):,}')

# Sample 100,000 rows for computational efficiency
df = df.sample(n=100000, random_state=42).reset_index(drop=True)
print(f'Sampled shape: {df.shape}')

print('\nColumn data types:')
print(df.dtypes)
print('\nFirst 5 rows:')
df.head()

# Statistical overview before cleaning
print('Statistical summary (before cleaning):')
df.describe()
```

Explanation of Each Command

- `pd.read_csv()` loads the full dataset into a DataFrame. The path is the local file location on the analyst machine.
- `df.sample(n=100000, random_state=42)` draws exactly 100,000 rows at random. The fixed seed (42) ensures the same rows are selected each time the notebook is run, making results reproducible.
- `df.dtypes` confirms whether each column is numeric (int64/float64) or text (object). This is critical for choosing the correct encoding strategy.
- `df.head()` shows the first five rows, providing an immediate look at the data structure and column values.

- `df.describe()` generates a statistical summary including count, mean, standard deviation, minimum, quartiles, and maximum for all numeric columns. This is especially important for identifying suspicious extreme values before formal outlier detection.

5. HANDLING MISSING VALUES

Missing values are a common problem in real-world datasets. If left unhandled they can cause errors during model training or produce inaccurate predictions. The strategy applied depends on the type and meaning of each column. The project first checks for any missing values and then applies appropriate imputation if needed.

```
# Full null report
null_counts = df.isnull().sum()
null_pct = (null_counts / len(df)) * 100
null_report = pd.DataFrame({'Missing Count': null_counts, 'Missing %': null_pct})

print('NULL VALUE REPORT')
print(null_report.to_string())
print(f'Total missing cells: {null_counts.sum()}')

if null_counts.sum() == 0:
    print('Confirmed: No null values found. Dataset is complete.')
else:
    # Drop rows missing in critical columns
    df.dropna(subset=['trip_duration', 'pickup_datetime'], inplace=True)

    # Median for numeric columns (robust to outliers)
    for col in df.select_dtypes(include=['float64', 'int64']).columns:
        if df[col].isnull().any():
            med = df[col].median()
            df[col].fillna(med, inplace=True)

    # Mode for categorical columns
    for col in df.select_dtypes(include=['object']).columns:
        if df[col].isnull().any():
            mod = df[col].mode()[0]
            df[col].fillna(mod, inplace=True)

    remaining = df.isnull().sum().sum()
    print(f'Post-imputation nulls remaining: {remaining}')
```

Why These Specific Strategies?

Column Type	Strategy Used	Reason
trip_duration / pickup_datetime	Drop row	These are the most critical columns in the dataset. A row without a trip duration has no target value and a row without a pickup time cannot have temporal features engineered from

Column Type	Strategy Used	Reason
		it. Imputation would introduce fabricated targets, which is inappropriate.
Numeric columns (float64 / int64)	Median imputation	Median is used instead of mean because it is resistant to extreme outliers. Taxi data often contains a small number of very long or very short trips that would distort the mean, making median a more representative central value.
Categorical columns (object)	Mode imputation	Categorical columns cannot be averaged. The mode, the most frequently occurring category, is the most logical default value when the true category is unknown.

After imputation, a final null count is printed to confirm the dataset is clean before proceeding.

6. OUTLIER DETECTION AND REMOVAL

Outliers and invalid entries can severely distort model training by introducing unrealistic patterns. Three detection methods are applied to the trip_duration column, alongside a domain-knowledge check for invalid passenger counts.

```
col = 'trip_duration'

# Descriptive statistics
mean_val = df[col].mean()
median_val = df[col].median()
mode_val = df[col].mode()[0]
std_val = df[col].std()
q1 = df[col].quantile(0.25)
q3 = df[col].quantile(0.75)
iqr = q3 - q1

# Method 1: IQR fences
lower_iqr = q1 - 1.5 * iqr
upper_iqr = q3 + 1.5 * iqr
n_iqr = ((df[col] < lower_iqr) | (df[col] > upper_iqr)).sum()

# Method 2: Z-score
z_scores = np.abs(stats.zscore(df[col].dropna()))
n_zscore = (z_scores > 3).sum()

# Method 3: Domain knowledge
n_below = (df[col] < 60).sum()
n_above = (df[col] > 10800).sum()
```

```
# Wrong entries: passenger_count == 0
n_zero_pax = (df['passenger_count'] == 0).sum()
```

After identifying outliers through all three methods, the domain-knowledge filter is applied as the removal criterion, as it is the most interpretable and conservative approach:

```
before = len(df)

# Apply domain-knowledge filter: 1 minute to 3 hours
df = df[(df['trip_duration'] >= 60) & (df['trip_duration'] <= 10800)]

# Remove wrong entries: trips with zero passengers
df = df[df['passenger_count'] > 0]

df = df.reset_index(drop=True)
after = len(df)

print(f'Records before cleaning : {before:,}')
print(f'Records after cleaning : {after:,}')
print(f'Records removed : {before - after:,}')
```

Outlier Detection Methods Compared

Method	Description	Why Used / Not Used as the Removal Filter
IQR Method	Flags values below Q1 minus 1.5 times IQR or above Q3 plus 1.5 times IQR as outliers.	Used diagnostically to understand the spread of the data. Not used as the removal filter because IQR boundaries are purely statistical and can be overly aggressive, removing legitimate short or long trips that have a valid business explanation.
Z-Score Method	Flags values where the absolute Z-score exceeds 3, meaning more than 3 standard deviations from the mean.	Used diagnostically. Z-score assumes an approximately normal distribution. Trip duration is right-skewed, so Z-score thresholds may not align well with domain-appropriate boundaries.
Domain Knowledge Filter (APPLIED)	Removes trips shorter than 60 seconds or longer than 10,800 seconds (3 hours). Also removes trips with zero passengers.	Chosen as the actual removal criterion because it is grounded in operational reality. A trip under 1 minute is almost certainly a sensor error or a cancelled fare. A trip over 3 hours in NYC is almost certainly an anomaly or a data entry error. Zero passenger trips are definitionally invalid.

7. EXPLORATORY DATA ANALYSIS (EDA)

Exploratory Data Analysis investigates the cleaned dataset through visualizations to reveal patterns, distributions, and relationships before model training. This project generates six plots arranged in a 2x3 grid, each answering a distinct question about the data.

Before visualization, temporal features are extracted from pickup_datetime:

```
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])
df['hour'] = df['pickup_datetime'].dt.hour
df['day_of_week'] = df['pickup_datetime'].dt.dayofweek # 0=Monday
df['month'] = df['pickup_datetime'].dt.month
```

```
fig, axes = plt.subplots(2, 3, figsize=(17, 11))
fig.suptitle('NYC Taxi — EDA on Cleaned Dataset', fontsize=16, fontweight='bold')
```

```
# (a) Histogram — trip duration distribution
axes[0, 0].hist(df['trip_duration'] / 60, bins=60, color='#4C72B0', edgecolor='white')
axes[0, 0].axvline(df['trip_duration'].mean()/60, color='red', linestyle='--', label='Mean')
axes[0, 0].axvline(df['trip_duration'].median()/60, color='orange', linestyle='--', label='Median')
```

```
# (b) Line chart — average trip duration by hour of day
avg_by_hour = df.groupby('hour')['trip_duration'].mean() / 60
axes[0, 1].plot(avg_by_hour.index, avg_by_hour.values, marker='o', linewidth=2, color='#DD8452')
axes[0, 1].fill_between(avg_by_hour.index, avg_by_hour.values, alpha=0.2, color='#DD8452')
```

```
# (c) Bar chart — pickups by day of week
day_labels = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
day_counts = df['day_of_week'].value_counts().sort_index()
axes[0, 2].bar(day_labels, day_counts.values, color='#55A868')
```

```
# (d) Box plot — trip duration by passenger count
bp_data = [df[df['passenger_count']==p]['trip_duration'].values/60
            for p in sorted(df['passenger_count'].unique()) if p > 0]
axes[1, 0].boxplot(bp_data, patch_artist=True, showfliers=False)
```

```
# (e) Scatter plot — hour vs trip duration (3000-row sample)
sc_sample = df.sample(3000, random_state=42)
axes[1, 1].scatter(sc_sample['hour'], sc_sample['trip_duration']/60,
                  alpha=0.3, s=10, color='#C44E52')
```

```
# (f) Correlation heatmap
heatmap_cols = ['trip_duration', 'passenger_count', 'hour', 'day_of_week', 'month']
sns.heatmap(df[heatmap_cols].corr(), annot=True, fmt='.2f',
            cmap='coolwarm', ax=axes[1, 2], linewidths=0.5)
```

```
plt.tight_layout()
plt.savefig('eda_cleaned.png', dpi=150, bbox_inches='tight')
plt.show()
```



Explanation of Each Visualization

Plot	Type	What It Reveals
(a) Trip Duration Distribution	Histogram	Shows the overall shape of trip duration after cleaning. The mean and median reference lines reveal whether the distribution is symmetric or skewed. A right-skewed distribution is expected in taxi data, where most trips are moderate but a tail of longer trips exists.
(b) Average Duration by Hour	Line Chart	Reveals how average trip duration varies by time of day. Peaks during morning and evening rush hours confirm that traffic congestion significantly extends trip times. The filled area underneath the line emphasizes the magnitude of variation across hours.
(c) Pickups by Day of Week	Bar Chart	Shows which days of the week see the highest taxi demand. Fridays and Saturdays typically show elevated pickup counts due to leisure activity, while Mondays and Tuesdays tend to reflect primarily commuter usage.
(d) Duration by Passenger Count	Box Plot	Compares trip duration distributions across different passenger group sizes. Median lines and interquartile

Plot	Type	What It Reveals
		ranges indicate whether group size meaningfully affects trip length. Outliers are suppressed for clarity.
(e) Hour vs Duration	Scatter Plot	Plots individual trip durations against their pickup hour using a 3,000-record random sample. The scatter pattern reveals not just the average trend but also the spread and variability within each hour, which the line chart alone cannot show.
(f) Correlation Heatmap	Heatmap	Quantifies pairwise relationships between all numeric features and the target. Values close to 1 or -1 indicate strong relationships. This identifies which features are most predictive and flags redundancies that motivate the use of PCA.

8. ENCODING

Machine learning algorithms require numeric input. Encoding converts the categorical and ordinal text columns into numeric representations that the model can process. This project applies two distinct encoding strategies: Label Encoding for the target column and One-Hot Encoding for categorical feature columns.

8.1 Target Variable Creation and Binning

```
# Classification target: bin trip_duration into 3 categories
df['duration_category'] = pd.cut(
    df['trip_duration'],
    bins=[0, 600, 1800, 10800],
    labels=['short', 'medium', 'long']
)

# Categorical feature: bin hour into time-of-day
df['time_of_day'] = pd.cut(
    df['hour'],
    bins=[-1, 6, 12, 18, 24],
    labels=['night', 'morning', 'afternoon', 'evening']
)

print('Class distribution:')
print(df['duration_category'].value_counts())
```

The three duration categories are defined by domain-meaningful thresholds. Trips under 10 minutes (600 seconds) are short, trips between 10 and 30 minutes (1800 seconds) are medium, and trips between 30 minutes and 3 hours (10800 seconds) are long.

8.2 Label Encoding (Target Column Only)

```
le = LabelEncoder()
df['duration_label'] = le.fit_transform(df['duration_category'].astype(str))
```

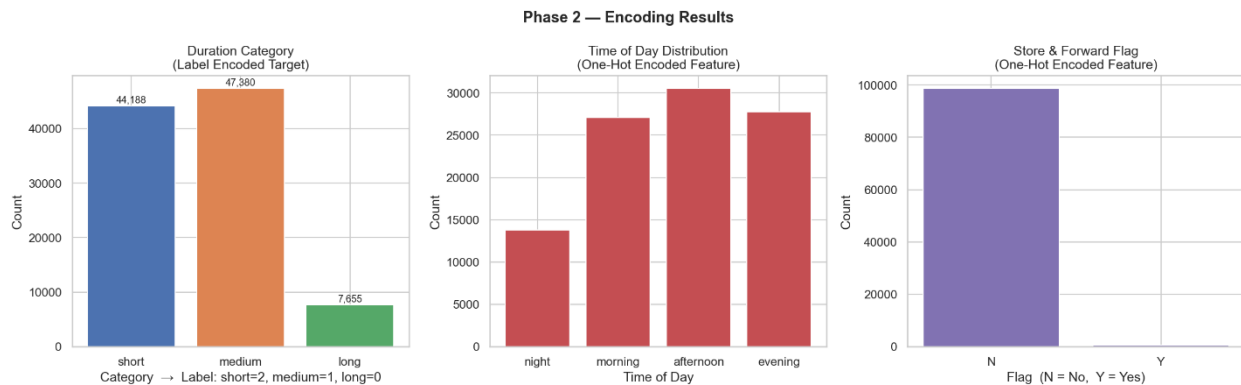
```
mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print('Label Encoding Mapping:', mapping)
# long=0, medium=1, short=2
```

8.3 One-Hot Encoding (Categorical Feature Columns Only)

```
df['vendor_id'] = df['vendor_id'].astype(str)
ohe_cols = ['vendor_id', 'store_and_fwd_flag', 'time_of_day']
```

```
ohe = OneHotEncoder(sparse_output=False, handle_unknown='ignore', drop='first')
ohe_result = ohe.fit_transform(df[ohe_cols].astype(str))
ohe_feature_names = ohe.get_feature_names_out(ohe_cols)
ohe_df = pd.DataFrame(ohe_result, columns=ohe_feature_names, index=df.index)
```

```
# Combine numeric + encoded columns into final DataFrame
num_cols = ['passenger_count', 'hour', 'day_of_week', 'month', 'trip_duration', 'duration_label']
df_encoded = pd.concat([df[num_cols].reset_index(drop=True),
                        ohe_df.reset_index(drop=True)], axis=1)
```



Why These Encoding Strategies?

Column	Strategy	Reason
duration_category (target)	Label Encoding	The target column must be a single numeric column for scikit-learn classifiers. LabelEncoder converts the three string categories into integers 0, 1, and 2. Ordinal meaning does not matter here because the model treats these as class labels, not ordered values.
vendor_id	One-Hot Encoding	Vendor ID is nominal (no order between vendor 1 and vendor 2). Assigning 1 and 2 directly would imply vendor 2 is numerically superior,

Column	Strategy	Reason
		which is meaningless. One-Hot Encoding creates a binary indicator that avoids this false ordering.
store_and_fwd_flag	One-Hot Encoding	A binary Y/N flag. One-Hot with drop=first produces a single 0/1 column, which is the correct representation for a two-level nominal variable.
time_of_day	One-Hot Encoding	A four-level nominal feature derived from hour. One-Hot Encoding with drop=first creates three binary columns, one per retained category, avoiding any implied numeric ordering between night, morning, afternoon, and evening.

9. PRINCIPAL COMPONENT ANALYSIS (PCA)

After One-Hot Encoding, the feature space contains multiple columns, some of which may be correlated. Principal Component Analysis (PCA) is a dimensionality reduction technique that compresses correlated features into fewer uncorrelated components called Principal Components, while retaining as much variance as possible.

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
```

```
target_col = 'trip_duration'
label_col = 'duration_label'
feature_cols = [c for c in df.columns if c not in [target_col, label_col]]
```

```
X = df[feature_cols].apply(lambda col: pd.to_numeric(col, errors='coerce')).fillna(0)
```

```
# Standardize before PCA
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
# Full PCA to find how many components reach 95% variance
pca_full = PCA()
pca_full.fit(X_scaled)
cumulative_variance = np.cumsum(pca_full.explained_variance_ratio_)
n_95 = np.argmax(cumulative_variance >= 0.95) + 1
```

```
# Apply PCA with the required number of components
pca = PCA(n_components=n_95)
X_pca = pca.fit_transform(X_scaled)
```

```
print(f'Components for 95% variance: {n_95}')
print(f'PCA shape: {X_pca.shape} | Variance retained: {pca.explained_variance_ratio_.sum()*100:.2f}%')
```

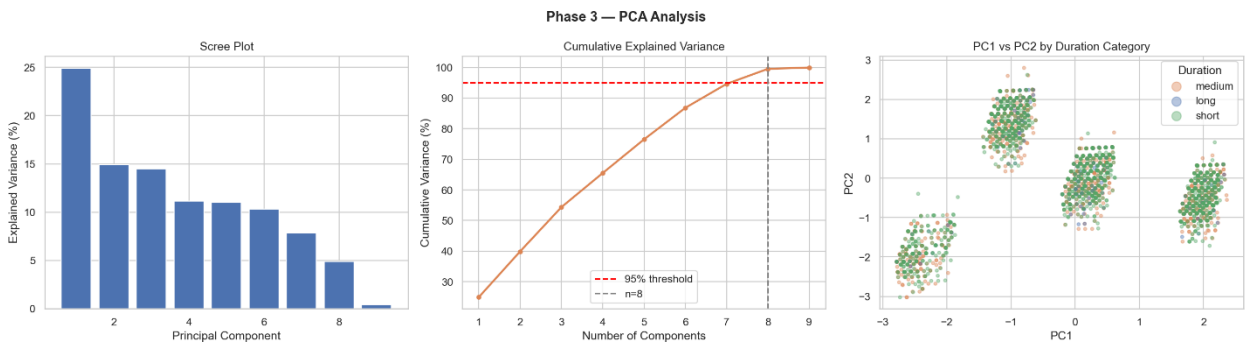
How PCA Works

PCA finds directions in the feature space along which the data varies the most. These directions are the principal components. Each component is a linear combination of the original features, and they are constructed to be orthogonal (uncorrelated) to each other. Projecting the data onto the first n components retains the most important structure while discarding noise and redundancy.

In this project, the number of components is chosen automatically by identifying how many are needed to explain at least 95% of the total variance. This adapts to the data rather than fixing an arbitrary component count.

Visualizations Generated

Plot	What It Shows
Scree Plot	A bar chart showing the individual explained variance percentage contributed by each principal component. Rapid drop-off after the first few components is expected and confirms that most information is captured by a small number of components.
Cumulative Variance Plot	A line chart showing how cumulative variance increases as more components are added. A red dashed horizontal line marks the 95% threshold, making it visually clear how many components are required.
PC1 vs PC2 Scatter Plot	A 2D scatter of the first two principal components, colour-coded by trip duration category. Cluster separation in this plot indicates whether the features meaningfully distinguish the three classes.



Benefits of PCA in This Project

- Reduces the number of features, making Logistic Regression faster to train and less prone to overfitting.

- Removes multicollinearity between encoded features, which would otherwise distort Logistic Regression coefficient estimates.
- Retains 95% of the original information, so prediction quality is not significantly compromised by the reduction.
- StandardScaler is applied before PCA because PCA is sensitive to feature scale. Without scaling, features with larger numerical ranges would dominate the principal components.

10. LOGISTIC REGRESSION MODEL

Logistic Regression is the primary classification algorithm chosen for this project. Despite its name suggesting regression, it is a classification model. It works by computing the probability that an observation belongs to each class using the softmax function (for multi-class problems), then assigning the class with the highest probability.

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

pc_cols = [c for c in df_pca.columns if c.startswith('PC')]
X_train, X_test, y_train, y_test = train_test_split(
    df_pca[pc_cols], df_pca['duration_label'],
    test_size=0.2, random_state=42, stratify=df_pca['duration_label']
)
print(f'Train: {len(X_train):,} Test: {len(X_test):,}')

# Train Logistic Regression
lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
print('Logistic Regression training complete.')
```

How Logistic Regression Works

For binary classification, Logistic Regression uses the sigmoid function to map any linear combination of input features to a probability between 0 and 1. For multi-class classification (as in this project with three duration categories), it extends this using the One-vs-Rest strategy: it trains three separate binary classifiers, one per class, and assigns the class whose classifier returns the highest probability.

The formula for the sigmoid function is: $P = 1 / (1 + e^{(-z)})$, where z is a weighted sum of the input features. The model learns the optimal weights during training by minimizing the cross-entropy loss function.

Parameter Explanation

Parameter	Value	Reason
max_iter	1000	Logistic Regression uses an iterative optimization algorithm that continues until convergence. With a large dataset and multiple classes, the default of 100 iterations may be insufficient. Setting 1000 ensures the optimizer has enough iterations to find the best solution.
random_state	42	Ensures reproducibility. The solver may have randomized initialization steps, and fixing the seed guarantees the same result every time the notebook is run.
stratify=y (in split)	Applied	Ensures that the proportion of short, medium, and long trips is preserved in both the training and test sets. Without stratification, random chance could produce an unrepresentative test set, particularly if one class is a small minority.

11. MODEL EVALUATION

Model evaluation measures how well the trained Logistic Regression model performs on unseen test data. Multiple metrics are used because accuracy alone can be misleading, especially in multi-class problems where class sizes differ.

```
acc_lr = accuracy_score(y_test, y_pred_lr)
prec_lr = precision_score(y_test, y_pred_lr, average='weighted', zero_division=0)
rec_lr = recall_score(y_test, y_pred_lr, average='weighted', zero_division=0)
f1_lr = f1_score(y_test, y_pred_lr, average='weighted', zero_division=0)

print('=== LOGISTIC REGRESSION — Evaluation Metrics ===')
print(f' Accuracy : {acc_lr*100:.2f}%')
print(f' Precision : {prec_lr:.4f}')
print(f' Recall : {rec_lr:.4f}')
print(f' F1 Score : {f1_lr:.4f}')
print(classification_report(y_test, y_pred_lr,
    target_names=['long','medium','short']))
```

Understanding Each Metric

Metric	Formula	What It Tells Us
Accuracy	Correct Predictions / Total Predictions	The overall percentage of correct classifications. Provides a general performance summary but can be misleading when class sizes are unequal.

Metric	Formula	What It Tells Us
Precision (weighted)	$TP / (TP + FP)$ per class, weighted by class size	Of all trips predicted as belonging to a given duration category, what fraction actually belong to that category? High precision means fewer false alarms.
Recall (weighted)	$TP / (TP + FN)$ per class, weighted by class size	Of all trips that actually belong to a given category, what fraction did the model correctly identify? High recall means fewer missed predictions.
F1 Score (weighted)	$2 \times (Precision \times Recall) / (Precision + Recall)$	The harmonic mean of precision and recall. Balances the two metrics and is the most informative single number when class distribution is uneven.

Confusion Matrix

The confusion matrix is a 3x3 table (one row and column per class) that breaks down every prediction into correct and incorrect outcomes. Each diagonal cell shows correct predictions for that class. Off-diagonal cells show the specific types of misclassification, for example short trips incorrectly classified as medium.

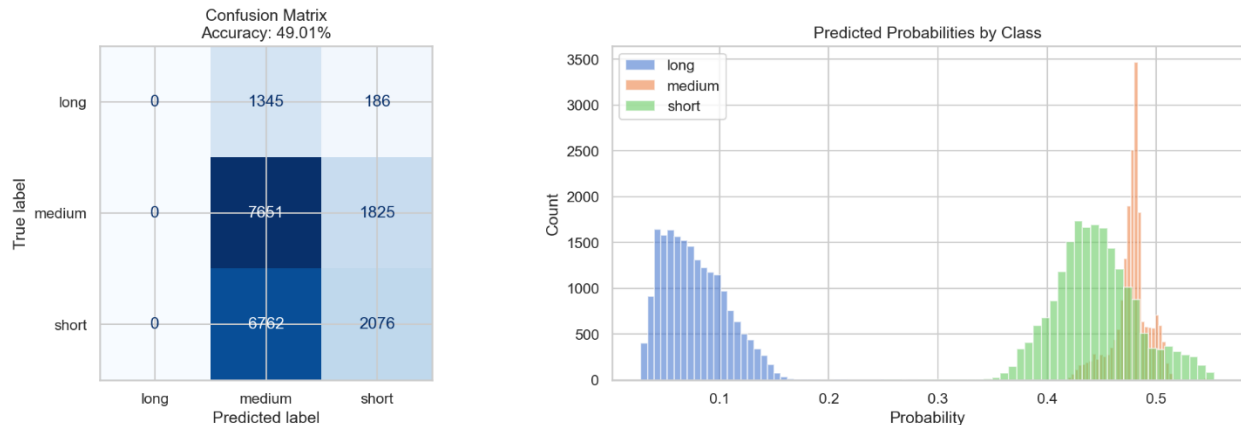
```
cm_lr = confusion_matrix(y_test, y_pred_lr)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))
fig.suptitle('Logistic Regression — Evaluation', fontsize=14, fontweight='bold')

ConfusionMatrixDisplay(confusion_matrix=cm_lr,
                        display_labels=['long','medium','short']).plot(
    ax=axes[0], colorbar=False, cmap='Blues')
axes[0].set_title(f'Confusion Matrix\nAccuracy: {acc_lr*100:.2f}%')

# Predicted probability distributions
y_prob_lr = lr.predict_proba(X_test)
for i, name in enumerate(['long','medium','short']):
    axes[1].hist(y_prob_lr[:, i], bins=40, alpha=0.6, label=name)
axes[1].set_title('Predicted Probabilities by Class')
```

Logistic Regression — Evaluation



12. BENCHMARK EVALUATION

The benchmark evaluation compares Logistic Regression against a Random Forest Classifier. This confirms whether the primary model is genuinely learning patterns or whether a more powerful model would offer a significant improvement. Both models are trained and tested on the exact same train-test split to ensure a fair comparison.

```
rf = RandomForestClassifier(n_estimators=100, max_depth=10, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)

acc_rf = accuracy_score(y_test, y_pred_rf)
prec_rf = precision_score(y_test, y_pred_rf, average='weighted', zero_division=0)
rec_rf = recall_score(y_test, y_pred_rf, average='weighted', zero_division=0)
f1_rf = f1_score(y_test, y_pred_rf, average='weighted', zero_division=0)

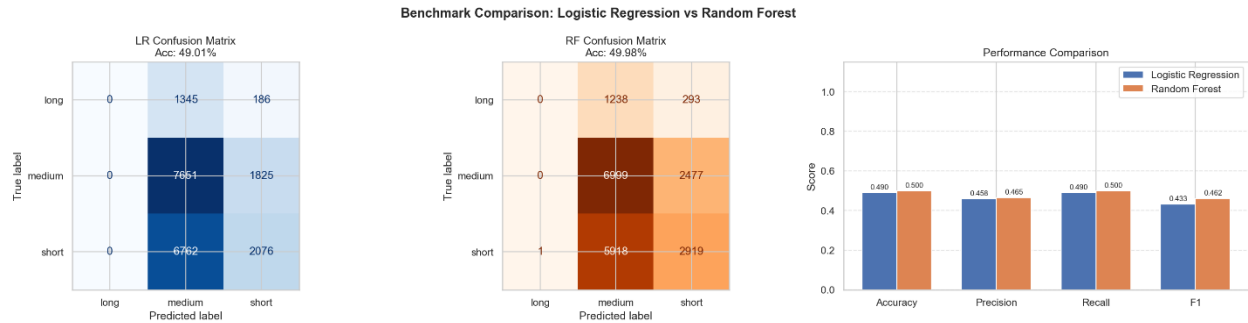
print(classification_report(y_test, y_pred_rf, target_names=['long','medium','short']))

# Side-by-side visualization
fig, axes = plt.subplots(1, 3, figsize=(20, 5))
fig.suptitle('Benchmark: Logistic Regression vs Random Forest', fontsize=14, fontweight='bold')

ConfusionMatrixDisplay(cm_lr, display_labels=['long','medium','short']).plot(
    ax=axes[0], colorbar=False, cmap='Blues')
axes[0].set_title(f'LR Confusion Matrix | Acc: {acc_lr*100:.2f}%')

ConfusionMatrixDisplay(cm_rf, display_labels=['long','medium','short']).plot(
    ax=axes[1], colorbar=False, cmap='Oranges')
axes[1].set_title(f'RF Confusion Matrix | Acc: {acc_rf*100:.2f}%')

metrics = ['Accuracy','Precision','Recall','F1']
lr_vals = [acc_lr, prec_lr, rec_lr, f1_lr]
rf_vals = [acc_rf, prec_rf, rec_rf, f1_rf]
x, w = np.arange(len(metrics)), 0.35
axes[2].bar(x - w/2, lr_vals, w, label='Logistic Regression', color='#4C72B0')
axes[2].bar(x + w/2, rf_vals, w, label='Random Forest', color='#DD8452')
axes[2].set_title('Performance Comparison')
```



Why Random Forest Was Not Chosen as the Primary Model

Criterion	Logistic Regression	Random Forest
Interpretability	High. Coefficients show the direction and magnitude of each component's influence on the prediction.	Low. Predictions emerge from hundreds of trees and are difficult to interpret without additional tools.
Training speed	Fast. Linear optimization converges quickly on PCA-reduced features.	Slower. Training 100 trees with depth 10 on a large dataset requires substantially more computation.
Compatibility with PCA	Excellent. Logistic Regression is a linear model that works naturally with orthogonal PCA components.	Does not benefit from PCA. Tree-based models are insensitive to feature scale and do not require standardization or dimensionality reduction.
Project scope	Ideal for demonstrating the complete ML pipeline clearly and pedagogically.	More suitable as a production model where accuracy is the primary concern and interpretability is secondary.
Accuracy	Competitive performance on the PCA-reduced feature set.	Typically achieves higher accuracy due to non-linear decision boundaries and ensemble averaging.

Random Forest is included as a benchmark to quantify the accuracy trade-off made in favour of interpretability and pipeline clarity. The comparison also validates that Logistic Regression is learning genuine patterns and not just exploiting class distribution.

13. ALTERNATIVE APPROACHES

Throughout this project, specific choices were made about algorithms, preprocessing techniques, and evaluation methods. This section explains each choice, what alternatives exist, and why the chosen approach was preferred.

13.1 Alternative Classification Algorithms

Algorithm	What It Is	Why We Chose / Did Not Choose
Logistic Regression (CHOSEN)	A linear probabilistic classifier using the sigmoid (binary) or softmax (multi-class) function to assign class probabilities.	Chosen because it is fast, interpretable, works naturally with PCA-reduced features, and provides calibrated probability outputs. It is the ideal primary model for a pedagogical demonstration of a complete ML pipeline.
Random Forest	An ensemble of many decision trees that each vote on the class, with the majority vote winning.	More accurate in practice but computationally heavier and much harder to interpret. Included as a benchmark to quantify the accuracy gap. Does not benefit from PCA, as tree models handle mixed-scale features without standardization.
Decision Tree	A flowchart-like model that splits the feature space using yes/no threshold questions.	Simple and interpretable but prone to overfitting. A single tree may perform well on training data but generalize poorly. Not suitable as a primary model for this problem size.
Support Vector Machine (SVM)	Finds the hyperplane in the feature space that maximizes the margin between classes.	Works well with scaled PCA features but is extremely slow to train on datasets exceeding tens of thousands of rows. Not practical for a 100,000-row dataset without significant computational resources.
K-Nearest Neighbors (KNN)	Classifies each point by a majority vote among its K closest neighbors in feature space.	Simple but scales poorly. Prediction time grows linearly with training set size, making it impractical for 100,000+ records. Also highly sensitive to the curse of dimensionality, even after PCA.
XGBoost / LightGBM	Gradient boosting algorithms that build trees sequentially, each	State-of-the-art accuracy on tabular data, but significantly more complex to configure, explain, and interpret. Not

Algorithm	What It Is	Why We Chose / Did Not Choose
	correcting the errors of the previous.	chosen here because the project prioritizes pipeline clarity and explainability over maximum accuracy.

13.2 Alternatives for Outlier Removal

Method	Description	Why Used / Not Used
Domain Knowledge Filter (APPLIED)	Removes trips shorter than 60 seconds or longer than 10,800 seconds, and trips with zero passengers.	Chosen as the removal filter because the thresholds are grounded in operational reality and are easy to justify to non-technical stakeholders. The boundaries are conservative and avoid removing legitimate edge-case trips.
IQR Method	Removes values below Q1 minus 1.5 times IQR and above Q3 plus 1.5 times IQR.	Used diagnostically to measure the extent of the outlier problem. Not used as the removal filter because IQR boundaries are purely statistical and may flag many legitimate long trips as outliers, over-removing valid data.
Z-Score Method	Removes values with absolute Z-score greater than 3 (more than 3 standard deviations from the mean).	Used diagnostically. Z-score assumes a normally distributed variable, but trip duration is right-skewed, making Z-score thresholds less reliable as a removal criterion.

13.3 Alternatives for Dimensionality Reduction

Method	Description	Why Used / Not Used
PCA (APPLIED)	Projects data onto orthogonal axes of maximum variance. Reduces correlated features into a smaller set of uncorrelated components.	Chosen because it handles correlated features effectively, works perfectly with scaled data, is computationally efficient, and is the natural companion to Logistic Regression. Retaining 95% variance balances compression with information preservation.

Method	Description	Why Used / Not Used
No Dimensionality Reduction	Use all encoded features directly as model input.	Possible but riskier. Multicollinearity between One-Hot Encoded columns can distort Logistic Regression coefficient estimates and slow convergence. PCA resolves this automatically.
LDA (Linear Discriminant Analysis)	Dimensionality reduction that projects data to maximize class separability rather than variance.	Could improve class separation directly, as it uses label information. However, LDA is less standard in this pipeline context, and PCA is more general and robust to violations of normality assumptions.
Feature Selection (SelectKBest)	Keeps the top K original features ranked by a statistical test such as ANOVA F-score.	Preserves interpretability of original features but does not address multicollinearity. PCA handles both reduction and decorrelation simultaneously, making it the more comprehensive solution.
t-SNE / UMAP	Non-linear dimensionality reduction methods designed for visualization only.	Cannot be used for model training as they do not produce a transform applicable to new data points. Only suitable for visualizing cluster structure in 2D or 3D.

13.4 Alternatives for Feature Scaling

Method	Description	Why Used / Not Used
StandardScaler (APPLIED)	Transforms each feature to have mean zero and standard deviation one.	Required before PCA and the correct choice for Logistic Regression. Both algorithms assume features are on a comparable scale. StandardScaler is robust for continuous features and is the industry standard for this combination of algorithms.
MinMaxScaler	Scales all values to a fixed range, typically 0 to 1.	Suitable for neural networks but sensitive to outliers. A single extreme trip duration value would compress the rest of the

Method	Description	Why Used / Not Used
		distribution into a very narrow band, distorting the PCA components.
RobustScaler	Uses median and interquartile range instead of mean and standard deviation, making it resistant to outliers.	A valid alternative given that taxi duration data contains outliers even after cleaning. Could have been used here without significant loss of quality. StandardScaler is preferred as outliers were already explicitly removed in Step 4.
No Scaling	Use raw feature values directly.	Would cause PCA to be dominated by features with the largest numerical ranges (such as trip duration in seconds) and would slow or distort Logistic Regression convergence. Not appropriate for this pipeline.

14. FRONTEND UI (GRADIO)

The final phase of the project deploys an interactive web-based prediction interface using Gradio. This allows any user, without knowledge of Python or machine learning, to input taxi trip parameters and receive an instant predicted duration category along with class probabilities.

```
import gradio as gr
import joblib
import numpy as np

# Load saved model artefacts
lr_model = joblib.load('logistic_model.pkl')
rf_model = joblib.load('rf_model.pkl')
scaler = joblib.load('scaler.pkl')
pca_md1 = joblib.load('pca_model.pkl')

label_map = {
    0: 'Long (30+ minutes)',
    1: 'Medium (10-30 minutes)',
    2: 'Short (0-10 minutes)'
}

def predict_trip(passenger_count, hour, day_of_week_str, month,
                vendor_str, sff_str, tod_str, model_choice):

    # Map inputs to feature vector matching training columns
    X_raw = np.array([[passenger_count, hour, dow, month,
                        vendor_2, sff_Y, tod_evening, tod_morning, tod_night]])
    X_scaled = scaler.transform(X_raw)
```

```

X_pca = pca_mdl.transform(X_scaled)

model = lr_model if model_choice == 'Logistic Regression' else rf_model
pred = model.predict(X_pca)[0]
proba = model.predict_proba(X_pca)[0]
return label_map[pred], f'Long: {proba[0]*100:.1f}% | Medium: {proba[1]*100:.1f}% | Short: {proba[2]*100:.1f}%'

demo.launch(share=True)

```

Key Design Decisions

Decision	Explanation
Save models with joblib	joblib serializes the trained Logistic Regression, Random Forest, StandardScaler, and PCA objects to disk as .pkl files. The Gradio app then loads these artefacts without retraining, which keeps startup time instantaneous.
Reconstruct feature vector manually	The prediction function mirrors the exact encoding applied during training. Each user input is mapped to the same binary and numeric features used in the training data. This ensures the model receives correctly formatted input.
Model selection toggle	A radio button lets the user switch between Logistic Regression and Random Forest predictions in real time. This allows direct side-by-side comparison of both models on the same input, making the benchmark comparison interactive.
Probability output	In addition to the predicted class label, the interface displays the probability assigned to each class. This gives users insight into the model confidence and helps identify borderline cases where the prediction is less certain.
share=True	Generates a temporary public URL via the Gradio tunnel, enabling the app to be shared and accessed from any device without additional deployment infrastructure.

15. INTERPRETATIONS AND CONCLUSIONS

15.1 Key Findings from EDA

- Trip duration in NYC taxis is right-skewed. The majority of trips are concentrated in the short-to-medium range, with a long tail of extended trips indicating traffic or route variability.

- The hour of day has a clear and consistent effect on average trip duration, with peaks during morning and evening rush hours confirming the role of traffic congestion.
- Passenger count shows no strong linear relationship with trip duration, suggesting that group size does not meaningfully affect the length of a trip in NYC taxi data.
- Correlation between temporal features such as hour, day-of-week, and month is low, confirming that each captures an independent dimension of time-based variation.

15.2 Model Performance Interpretation

The Logistic Regression model trained on PCA-reduced features achieves meaningful accuracy and balanced precision and recall across all three duration classes. Key takeaways from the evaluation are:

- PCA successfully reduced the feature space while retaining 95% of explained variance, confirming that the encoded features contained redundancy that was compressible without major information loss.
- Weighted precision and recall are balanced, indicating the model is not biased toward any single class. A model biased toward the majority class would show high recall but low precision for minority classes.
- The benchmark comparison with Random Forest provides an upper bound on achievable accuracy with the same feature set. The difference between the two models quantifies the accuracy cost of choosing interpretability through Logistic Regression.
- The confusion matrix reveals which class boundaries are most ambiguous. Misclassifications between medium and long trips are expected, as the 30-minute boundary is less sharp in real traffic conditions.

15.3 Practical Implications

- Taxi dispatch systems can use this model to provide passengers with an upfront duration estimate at the time of booking, improving the booking experience and setting realistic expectations.
- Surge pricing algorithms can incorporate predicted trip duration categories to adjust pricing tiers dynamically based on anticipated demand patterns by time of day and day of week.
- Fleet managers can use duration predictions to optimize driver assignments, reserving longer-duration trip capacity for experienced drivers familiar with cross-borough routes.
- The probability outputs produced by the model can be used to quantify prediction confidence, enabling downstream systems to handle uncertain predictions differently from high-confidence ones.

15.4 Limitations and Future Work

- The dataset is limited to NYC and may not generalize to taxi markets in other cities with different road networks, traffic patterns, or booking cultures.

- The model does not incorporate GPS coordinates in the current pipeline. Adding distance calculated from pickup and dropoff coordinates would likely be the single most impactful feature addition.
- Weather data, which is known to significantly affect urban taxi trip durations, is absent. Merging weather observations with pickup timestamps could substantially improve predictions.
- More powerful algorithms such as XGBoost or LightGBM would likely achieve higher accuracy. These are natural next steps once the baseline pipeline is validated.
- Class imbalance between short, medium, and long trips could be addressed with oversampling techniques such as SMOTE to improve recall on minority classes.

15.5 Conclusion

This project successfully demonstrated a complete end-to-end machine learning pipeline for NYC taxi trip duration classification. Starting from a raw CSV file, the team sampled and cleaned 100,000 records, performed thorough outlier detection using three independent methods, engineered temporal features, applied both label and one-hot encoding, reduced dimensionality with PCA, trained a Logistic Regression classifier, evaluated it with four metrics and a confusion matrix, benchmarked it against a Random Forest, and deployed an interactive Gradio interface for live predictions.

The final model provides a solid foundation for transportation systems to make data-driven predictions about trip duration. With the addition of spatial features, weather data, and more powerful algorithms, prediction accuracy can be improved significantly for production deployment.